

THE UNIVERSITY OF NEW SOUTH WALES



08 - Design and Implementation of API Publication tool and schema mapping in E-invoicing API Ecosystem

Project Proposal

COMP9900 Computer Science Project

Term 2, 2026

26T2-9900-H19B-BREAD

Name	zID	Email	Role
Yiyang Chen	z5508707	z5508707@ad.unsw.edu.au	Product Owner, Back-end Developer
Junjie Zhang	z5557718	z5557718@ad.unsw.edu.au	Back-end Developer
Lin Wang	z5579191	z5579191@ad.unsw.edu.au	Scrum Master, Front-end Developer
Xincheng Hao	z5597528	z5597528@ad.unsw.edu.au	Back-end Developer, Database Engineer
Peishuo Wang	z5601881	z5601881@ad.unsw.edu.au	Back-end Developer
Xinyi Zhang	z5608247	z5608247@ad.unsw.edu.au	Product Manager, QA Engineer

20 June 2026

Table of Contents

- 1. Background 1**
 - 1.1. Problem Definition and Impact 1
 - 1.2. Review of Existing Systems 4
- 2. User Stories and Sprints 6**
 - 2.1. Scope Statement 6
 - 2.2. Sprint 1 User Stories 6
 - 2.3. Product Backlog 9
 - 2.4. Sprint Timeline and Milestones 11
- 3. Technical Design 14**
 - 3.1. System Architecture Diagram 14
 - 3.2. Interface storyboarding 15
 - 3.3. Design Justifications 23
 - 3.4. Functionality Mapping 28
- Bibliography 32**
- Appendix A: Scope Approval Evidence 33**

1. Background

1.1. Problem Definition and Impact

1.1.1. Problem Domain and Specific Problem

E-invoicing has become a primary part of modern-day tax compliance, public procurement and digitisation of the entire supply chain. Within the continual progression of digital business, e-invoicing is an effective way for both private sector companies as well as Government agencies to support the capture, creation, transmission, validation and storage of invoice-based transactional data through the exchange of electronic documents that are structured, standardised and machine-readable (Bellon et al., 2022).

The project will discuss how businesses are able to use the current available capabilities they have for creating, transmitting, validating and storing invoices through integration via APIs and supply chain systems. Companies now have to integrate their ERP systems, legacy systems, custom systems, and third-party e-invoicing services via APIs; however, without an established set of API publishing and documentation standards, this is becoming increasingly complex due to the fragmented nature of the enterprise environment and the requirement for consistent implementations of e-invoice standards across all systems and technical assumptions.

The Australian Taxation Office (ATO) is the Peppol Authority for Australia and therefore is responsible for setting local Peppol requirements and for managing Peppol participants in Australia. OpenPeppol provides further details surrounding localisation through PINT A-NZ Billing for Australia and New Zealand (*Australian Peppol Authority | ATO Software Developers*, n.d.). However, the local context also illustrates that the ability to adopt e-invoicing remains contingent on the readiness of the software as well as the ability of enterprise systems to integrate in a flexible manner. Based on a study conducted by the Australian Treasury, the adoption of Peppol eInvoicing by the private sector is quite limited, as only around 10,000 businesses out of an estimated 2.4 million businesses in Australia had registered for Peppol eInvoicing at the time of the consultation (*Supporting Business Adoption of Electronic Invoicing - Consultation Paper*, n.d.).

Another instance of regional fragmentation is demonstrated through the use of proprietary EDI systems and privately owned trading networks. The Australian Treasury pointed out that many large companies across a variety of sectors have invested heavily in building specially designed EDI systems to accommodate the needs of particular companies or trading networks. These proprietary solutions may act as “islands of trade” or closed trading spaces, and as a result, a large number of these systems do not directly support Peppol eInvoicing or communicate with other EDI systems (*Supporting Business Adoption of Electronic Invoicing - Consultation Paper*, n.d.). Additionally, as different teams are managing the internal APIs that link their enterprise systems with eInvoicing processes, there is often very little consistency in the manner in which APIs are defined, developed and documented.

As a result, it is difficult to identify or re-use eInvoicing APIs. Also, as many suppliers of eInvoicing have poorly documented APIs, inconsistent formats for receiving and returning data, together with poorly defined authentication criteria, there are likely to be multiple versions of eInvoicing data

structures or schemas provided by different suppliers, such as JSON payloads, UBL XML or other proprietary formats, without mapping to or validating against these formats and versions, leaving few APIs that can be reliably re-used for eInvoicing.

Thus, the primary limitation is not simply the absence of eInvoicing APIs. There is a severe lack of standardisation across existing API offerings for e-invoicing due to the fact that these API offerings exist across different systems and different system teams. As a result, developers working with e-invoices are generally forced to rely on “tribal knowledge” in order to properly use these API offerings rather than utilising standardised tooling.

1.1.2. Current State and Desired State

Developers working with e-invoices generally have no option but to rely on either inadequately documented or organised formal documentation on API offerings, documentation shared by different members of the development team, or attempts to understand an API through source code and other developer documentation. This is not only an issue from a local project perspective, but is also indicative of a broader problem with regard to the general API engineering community. For example, (Khan et al., 2021) identified that 95.2% of surveyed professional developers reported that API documentation smells negatively affect productivity. As noted by (Uddin et al., 2019), API users often seek different opinions or additional support around their API selections and decision-making processes regarding API reusability.

As developers use an e-invoicing API, it is critical for them to understand API specifications, methods of authenticating access to an API, supported invoicing standards, API schemas and associated mapping to an e-invoice. Due to the aforementioned challenges, along with the absence of a common machine-readable format, developers often have to depend on their or their team’s knowledge of how the API works.

The aim of this project is to establish a standardised and controlled publishing process for electronic invoice (e-invoice) application programming interfaces (APIs) by providing a means for enterprise users to submit e-invoice API specifications and metadata to a centralised repository of e-invoice APIs. E-invoice APIs that have been approved for inclusion in the repository will be reviewed, documented, validated, and developed in accordance with schema-aware standards. In turn, users will have improved access to well-documented and reliable e-invoicing APIs, thereby facilitating downstream service discovery and service composition. The impact of this will be an increase in API reuse, decreased duplication of integration efforts, and a solid base to move forward with e-invoicing operations.

1.1.3. Severity and Impact of the Problem

The business impact of this concern can be categorized into four major domains: tax compliance and digital reporting, operational efficiency, documentation and discoverability of APIs, and the governance risks of establishing an API repository for e-invoicing.

According to the European Commission, the VAT compliance gap within the European Union (EU) for 2023 was estimated to be €128 billion, or 9.5% of the VAT Total Tax Liability (*VAT Gap*

- *Taxation and Customs Union - European Commission*, n.d.). The size of that total is indicative of the problem with tax compliance that digital reporting initiatives are trying to address. Although e-invoicing will not completely eliminate tax reporting problems or VAT fraud, empirical studies have shown that e-invoicing can enhance the transparency of transactional data and help companies report more accurately. (Bellon et al., 2022) reported an increase in sales, purchases, and VAT liabilities for companies in Peru after their implementation of an e-invoice system for VAT. (Kotsogiannis et al., 2025) also researched the links between e-invoicing, tax audits, and VAT compliance. As a result, the intent of the project's work is not to offer an actual solution to tax compliance challenges but rather assist with one of the technological enablers of those direct solutions through the creation of structured, validated, and reusable Electronic Invoice APIs to support the e-Invoicing industry.

Operational and implementation expenses incurred by enterprises when implementing e-Invoicing also represent a significant challenge for enterprises. As outlined in the APEC e-Invoicing Interoperability Report in 2025, the average domestic e-invoicing implementation cost for large enterprises was USD 105,754, compared to an average of USD 36,016 for smaller enterprises and USD 4,850 for micro enterprises (*APEC 2025, Interoperability of Electronic Invoicing Systems in the APEC Region, Asia-Pacific Economic Cooperation*, n.d.). These numbers indicate that larger enterprises tend to have additional impediments to implementing e-Invoicing solutions when compared to smaller enterprises, owing to the complexity of their respective internal environments. If an enterprise's internal business unit APIs, including enterprise resource planning (ERP), service providers, validation tools, and archiving workflows are inadequately documented and/or difficult to integrate, an enterprise will likely not realise the operational and compliance benefits associated with e-Invoicing.

Similarly, API discoverability and documentation are also industry-wide challenges, as evidenced by the results from the 2025 Postman API report wherein 55% of survey participants indicated having problems with inconsistent API documentation, and 34% stated that they were unable to find existing APIs (*2025 State of the API Report | Postman*, n.d.). Research in the academic literature concerning API documentation and API usage decision making is consistent with this observation. (Khan et al., 2021) have found that API documentation 'smells' can negatively affect developer productivity and (Uddin et al., 2019) found that developers often have a need for additional information when attempting to determine whether an API can be reused effectively. These demonstrate that if APIs are not described using consistent and machine-readable representations, they cannot be discovered, combined, validated, mapped, or reused reliably.

The stakeholders impacted by these challenges include many different audiences. To manage the risk associated with the existing e-invoicing API ecosystem, reusability of APIs is critical; however, because APIs have inconsistent documentation and continue to involve reliance on the original implementation team's knowledge base, compliance teams are often unable to validate, evaluate or verify if the existing APIs meet the criteria necessary to be compliant with current security standards, authentication standards, data interchange standards, or available legislation. Additionally, insufficient corporate governance practices can result in challenges for downstream services attempting to search for, recommend or aggregate APIs with an adequate level of assurance.

In addition, ongoing fragmentation and degradation of the API governance processes will continue to hinder cost reductions associated with adopting new standards and negatively impact operations owing to duplicated integration activities.

1.1.4. Brief Proposed Solution Summary

To address the issues above, it is suggested that an API Publisher be utilised as an access portal and quality gate to e-Invoicing APIs. The API Publisher will include a repository-level API integration process, a structured process for submission of APIs, a metadata capture process, validation of submitted APIs and schema mapping rules before published APIs are discoverable and/or consumable from existing e-Invoice APIs.

By standardising how APIs are submitted, described, validated and made reusable, the API Publisher will help reduce the fragmentation and improve the reliability of the overall e-Invoicing API ecosystem.

1.2. Review of Existing Systems

1.2.1. Introduction & API Ecosystem Overview

In modern corporate environments, application programming interfaces (APIs) serve as the primary enablers for enterprise digital transformation, cross-platform infrastructure, and supply chain integration (De, 2017). Nevertheless, fragmentation in the technical ecosystem of e-invoicing is usually extremely high. Different operational departments within an enterprise, as well as different legacy ERP systems and vendors' applications, have been using different design patterns, conventions, and assumptions about how developers will write code when designing ways to generate invoices.

According to global architectural best practices, successful digital transformation requires large organizations to transition from informal "tribal knowledge" toward robust API governance, where interfaces are managed strictly as standard enterprise products (De, 2017). Without a controlled, automated API publishing tool to serve as an organizational quality gate, overall compliance risks escalate, and downstream service composition becomes impossible due to a lack of machine-readable specifications (De, 2017). To unpack this issue further, the following segment reviews two popular commercial tools commonly deployed to resolve pieces of the outlined challenge. These solutions include SwaggerHub for team-based API development work, alongside Apicurio Registry, which handles the oversight of data schemas.

1.2.2. Investigation of Existing Systems

1.2.2.1. System 1: SwaggerHub (API Design and Lifecycle Management Platform)

SwaggerHub is a cloud-based service developed by SmartBear around the OpenAPI Specification (OAS) to support the design-first approach to API development. Teams that have used traditional approaches to API design and development typically face three challenges: their interface descriptions do not follow consistent standards, their written documentation does not match the code that was

ultimately developed, and the workload of each team is duplicated due to a lack of collaboration between teams. Using SwaggerHub, teams can work on designing all of their APIs from the very beginning through a single shared workspace. This allows teams to establish formal API contracts before any development happens. The combination of the complete set of visual editing tools, live syntax validation, and auto-generated mock services ensures that all content created is consistent with OpenAPI guidelines. All of the technical specifications for the project are stored in machine readable files prior to any development beginning, so that all engineering teams working on the project can ensure that consistent API interfaces are maintained through the entire development process.

Limitation in E-Invoicing: SwaggerHub only generates static universal interface contracts and cannot accommodate unique e-invoicing rules set by specifications including PEPPOL and UBL 2.1. It can output JSON or YAML files free of syntax errors, yet these outputs lack functions to check internal financial details. There is no way to confirm cross-border tax separation data inside the file, or map these fiscal entries to the dedicated business rules defined in the document.

1.2.2.2. System 2: Apicurio Registry (Cloud-Native Schema and API Registry)

Developed as an open-source storage solution for schemas and API definitions, Apicurio Registry caters to the operational needs of cloud-native environments and asynchronous data processing setups. Most contemporary application infrastructures adopt unified registry platforms, which simplify ongoing management of dynamically updated data schemas running across separate microservice instances. Apicurio serves as this centralized store for data models (including OpenAPI, AsyncAPI, and JSON Schema), providing a mechanism to decouple data consumers from unintentional breaking changes caused by uncoordinated schema modifications. It enforces real-time compatibility checks (backwards, forwards, and full compatibility), enabling a heterogeneously distributed microservice architecture to dynamically retrieve schema definitions at runtime without service interruption.

Limitations in E-Invoicing: Traditional schema registries primarily function as passive storage facilities incapable of active transformations. Based on recent research, traditional registries face significant structural and semantic challenges when attempting to manage complex schema evolutions, such as converting data types across differing platforms (Fu & Chen, 2024). In an e-invoicing environment, supplier invoices typically contain proprietary payloads that must be actively converted into an official XML/UBL structure based on buyer requirements. Apicurio Registry cannot automatically map or transform these disparate data items (Fu & Chen, 2024). Furthermore, it lacks a built-in active schema translation engine necessary to bridge the structural divide between disparate invoice data models (Fu & Chen, 2024).

1.2.3. Future Evolution & Implications

The techniques used in this project show that a successful e-invoicing API ecosystem includes the use of standardised and automated schema alignment and a quality gate defining the design stage. In order to minimise the structural and semantic limitations caused by a purely passive registry, it is recommended that an API Publishing Tool with a Visual Schema Mapping Canvas and a Domain Compliance Pipeline be used. Finally, the creation of a central repository where all standardised, machine-readable OpenAPI specifications will be kept provides a critical resource for

future development of intelligent applications downstream. By capturing rich metadata at the point of publishing, this ecosystem enables seamless integration with state-of-the-art Large Language Models (LLMs) for the automatic generation of task-focused dialogue systems, as well as the capability to “hot-swap” services based solely on their definitions (Rodríguez-Sánchez et al., 2025). With these structural improvements put in place, governments and service operators can roll out three key digital tools within public service ecosystems: automated customer response pipelines, AI-powered invoice validation processes, and self-service access portals for technical developers.

2. User Stories and Sprints

2.1. Scope Statement

This project will deliver an API Publisher Tool for the e-invoicing API ecosystem. The system will allow enterprise publishers to authenticate, submit OpenAPI/Swagger or WSDL specifications, capture structured metadata, validate API definitions and compliance information, manage drafts and resubmissions, and publish approved APIs to a Central API Repository. It will also support published API listing, update and withdrawal, schema mapping, and integration with downstream Discovery and Composition services. Out of scope are real invoice transaction processing, production deployment, and full integration with external government systems.¹

2.2. Sprint 1 User Stories



H19BBREA Sprint 1 19 Jun – 3 Jul (6 work items)		25	0	Complete sprint	...
☒ H19BBREAD-1	FR-1 Publisher authentication and authorization	IN PROGRESS	5		
☒ H19BBREAD-2	FR-2 API specification upload	IN PROGRESS	5		
☒ H19BBREAD-3	FR-3 Metadata capture form	IN PROGRESS	3		
☒ H19BBREAD-4	FR-4 Specification validation	IN PROGRESS	8		
☒ H19BBREAD-5	FR-7 Validation feedback	IN PROGRESS	3		
☒ H19BBREAD-6	FR-9 Draft and resubmission support	IN PROGRESS	5		

Figure 1: Sprint 1 User Stories

2.2.1. H19BBREAD-1 FR-1 — Publisher Authentication and Authorization

Priority: High | **Story Points:** 5

As an enterprise publisher, I want to authenticate before accessing the API Publisher tool, so that only authorized enterprise users can submit, modify, or withdraw e-invoicing APIs.

Acceptance Criteria:

- Users are required to log in before being able to access the API Publisher tool.
- Only users who have authenticated with the system will be able to access the publishing functions (create, edit or delete) of API submissions.
- The system provides role-based access control for the creation, modification, and deletion of APIs.
- Users without appropriate authentication cannot access restricted functions of the API Publisher tool.

¹Client scope approval evidence as shown as Figure 9 is included in Appendix A.

- A clear message will be displayed to the user when their access has been denied due to lack of sufficient authorization.

2.2.2. H19BBREAD-2 FR-2 — API Specification Upload

Priority: High | **Story Points:** 5

As an enterprise publisher, I want to upload or provide a URL for an API specification, so that I can submit my REST or SOAP e-invoicing API for validation and publication.

Acceptance Criteria:

- The publisher can upload an OpenAPI/Swagger Specification for REST APIs.
- The publisher can upload a WSDL Specification for SOAP APIs.
- The publisher can provide a direct URL link to the hosted API Specification.
- The system provides an error message when an unsupported file format is attached.
- The system notifies the publisher that the API Specification was successfully uploaded or linked.
- The uploaded or linked specification is stored as part of the API submission record.

2.2.3. H19BBREAD-3 FR-3 — Metadata Capture Form

Priority: High | **Story Points:** 3

As an enterprise publisher, I want to enter structured business and technical metadata for my API, so that the API can be accurately validated, discovered, and used by downstream services.

Acceptance Criteria:

- A standardized metadata collection process is provided when an API is submitted.
- Mandatory items collected include: API name, API endpoint URL, protocol type, input data format, output data format, and authentication method.
- Publishers can provide a brief description of how the API functions.
- Publishers can select one or more e-invoicing capabilities to which the API will conform (e.g. invoice creation, validation, transmission, or archive).
- Mandatory fields are clearly identified, preventing submissions from being processed without the required metadata.
- All metadata provided is saved to the API submission record.

2.2.4. H19BBREAD-4 FR-4 — Specification Validation

Priority: High | **Story Points:** 8

As an enterprise publisher, I want the system to validate my uploaded API specification, so that invalid OpenAPI or WSDL documents can be detected before publication.

Acceptance Criteria:

- Before publication, the system validates OpenAPI/Swagger specifications against the relevant OpenAPI standard.

- Before publication, the system validates WSDL specifications against the relevant SOAP/WSDL structure.
- The system detects syntax errors, missing required elements, missing element types, and structural inconsistencies.
- Invalid specifications are rejected before they can enter the publication stage.
- The system generates a detailed error report when specification validation fails.
- Valid specifications can proceed to the next step of the submission workflow.

2.2.5. H19BBREAD-5 FR-7 — Validation Feedback

Priority: High | **Story Points:** 3

As an enterprise publisher, I want to receive clear validation feedback after submitting an API, so that I can understand which validation stages passed or failed and correct any issues.

Acceptance Criteria:

- The system provides validation results in a structured and easy-to-understand format.
- The validation results clearly identify which stages passed or failed.
- The system provides a reason for each failed validation stage.
- The system highlights issues that need to be corrected before resubmission.
- After viewing the validation results, the publisher can return to the submission workflow.
- The feedback is specific enough to help the publisher correct issues and resubmit the API.

2.2.6. H19BBREAD-6 FR-9 — Draft and Resubmission Support

Priority: Medium | **Story Points:** 5

As an enterprise publisher, I want to save incomplete API submissions as drafts and resubmit corrected APIs after validation failures, so that I do not need to re-enter all information from the beginning.

Acceptance Criteria:

- The system allows the publisher to save an incomplete API submission as a draft.
- Draft submissions retain uploaded specification information and entered metadata, where available.
- The publisher can open and edit a saved draft submission.
- If validation fails, the publisher can modify the existing submission and resubmit it.
- Previously entered metadata is retained when the submission is resubmitted.
- The system updates the submission status to reflect its current state (e.g. draft, rejected, or resubmitted).

2.3. Product Backlog

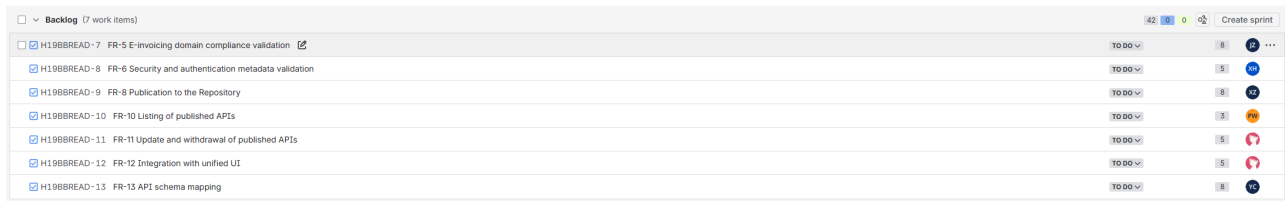


Figure 2: Product Backlog

2.3.1. H19BBREAD-7 FR-5 — E-Invoicing Domain Compliance Validation

Priority: High | **Story Points:** 8

As an enterprise publisher, I want the system to validate my submitted API against e-invoicing domain rules, so that only APIs consistent with recognised e-invoicing standards can be accepted into the repository.

Acceptance Criteria:

- The system verifies that API submissions are validated according to e-invoicing domain compliance rules.
- The system verifies the input, output, and operations of each API against recognised e-invoicing standards.
- The system identifies APIs that are not compliant with domain compliance rules.
- APIs that do not pass e-invoicing compliance are rejected prior to publication.
- When an API fails compliance validation, the system produces a domain-focused error report.
- After an API passes domain compliance, it may proceed to the next validation or publication stage.

2.3.2. H19BBREAD-8 FR-6 — Security and Authentication Metadata Validation

Priority: High | **Story Points:** 5

As an enterprise publisher, I want the system to validate the security and authentication metadata of my API, so that APIs without acceptable or complete security information cannot be published.

Acceptance Criteria:

- The system verifies that each API submission declares an acceptable authentication mechanism.
- Accepted authentication methods include OAuth 2.0, API Key, Basic Authentication, and mTLS.
- The system verifies that all security-related metadata has been specified in the API specification.
- The system verifies that authentication information is internally consistent with the submitted specification and metadata.
- APIs that do not meet the required security baseline are automatically rejected.
- If security or authentication metadata fails validation, the user receives clear messaging explaining the error.

2.3.3. H19BBREAD-9 FR-8 — Publication to the Repository

Priority: High | **Story Points:** 8

As an enterprise publisher, I want successfully validated APIs to be published to the Central API Repository, so that they can be discovered and used by downstream Discovery and Composition services.

Acceptance Criteria:

- The Central API Repository only accepts an API once all required validation stages have been completed successfully.
- The API specification and associated metadata are recorded in the system.
- The API record is stored in a format compatible with the Discovery Service.
- The API record is stored in a format compatible with the Composition Service.
- The system notifies the publisher when the API has been successfully published.
- If publication fails, an error message is displayed and the API is not marked as published.

2.3.4. H19BBREAD-10 FR-10 — Listing of Published APIs

Priority: Medium | **Story Points:** 3

As an enterprise publisher, I want to view a list of APIs that my enterprise has published, so that I can monitor their current status and manage them when needed.

Acceptance Criteria:

- Publishers can view the APIs that their enterprise has published using the API Publisher tool.
- Each API is displayed with key details: API name, protocol type, endpoint URL, and status.
- The system shows the current status of each API (e.g. published, rejected, or draft).
- The system provides an overview of all APIs registered by the publisher's enterprise.
- The API Publisher tool only shows APIs belonging to the publisher's registered enterprise.
- The API list is presented in a visually clear and readable format.

2.3.5. H19BBREAD-11 FR-11 — Update and Withdrawal of Published APIs

Priority: Medium | **Story Points:** 5

As an enterprise publisher, I want to update or withdraw a previously published API, so that outdated or incorrect APIs can be corrected or removed from the repository.

Acceptance Criteria:

- The publisher can select a previously published API for update.
- The publisher can update the API metadata or submit a revised version of the API specification.
- Updated APIs are subject to revalidation before being republished.
- The publisher can withdraw an API from the repository.
- Withdrawn APIs are no longer available for discovery or composition.
- The publisher receives a confirmation message when an API is successfully updated or withdrawn.

2.3.6. H19BBREAD-12 FR-12 — Integration with Unified UI

Priority: Medium | **Story Points:** 5

As an enterprise publisher, I want to access the API Publisher tool from the unified ecosystem interface, so that I can use publishing, discovery, and composition services within one consistent platform. **Acceptance Criteria:**

- The Publisher tool can be accessed through the unified ecosystem user interface.
- The Publisher interface uses consistent navigation with the Discovery Service and Composition Service.
- The Publisher interface is visually and interactively consistent with the rest of the platform.
- Users can move between publishing, discovery, and composition services without needing separate systems.
- The unified interface clearly identifies where the Publisher section is located.
- The unified user interface provides an integrated user experience across the ecosystem.

2.3.7. H19BBREAD-13 FR-13 — API Schema Mapping

Priority: Medium | **Story Points:** 8

As an enterprise publisher, I want the system to support schema mapping between APIs and additional data formats, so that API inputs and outputs can be aligned for downstream discovery and composition.

Acceptance Criteria:

- The system supports mapping one API schema to another API schema.
- The system identifies relevant input and output schema fields for a given mapping.
- Additional data formats can be supported within the schema mapping framework.
- Future enhancements can include format mappings such as PDF-to-XML mapping.
- Mapped schema information is stored in a format that can support downstream services.
- The system provides clear notifications when fields cannot be matched or mapped to the target schema.

2.4. Sprint Timeline and Milestones

The project is planned using a three-sprint structure aligned with the course schedule. Sprint 1 establishes the foundation of the API Publisher Tool, Sprint 2 focuses on major validation and repository integration, and Sprint 3 focuses on final integration, testing, refinement, and project completion. The internal deadlines below are team-planning checkpoints designed to leave time for integration testing, bug fixing, demo preparation, and sprint review.

2.4.1. Sprint 1

- **Calendar Dates:** 18 June 2026 - 2 July 2026
- **Main Focus:** Foundation and core API submission workflow.

- **User Stories / Backlog Focus:** In scope for Sprint 1: H19BBREAD-1 / FR-1 Publisher authentication and authorization; H19BBREAD-2 / FR-2 API specification upload; H19BBREAD-3 / FR-3 Metadata capture form; H19BBREAD-4 / FR-4 Specification validation; H19BBREAD-5 / FR-7 Validation feedback; H19BBREAD-6 / FR-9 Draft and resubmission support.
- **Internal Deadlines :**
 - 20 June: confirm Sprint 1 ownership and task breakdown.
 - 24 June: UI flow and backend skeleton completed.
 - 27 June: authentication, upload, and metadata capture draft completed.
 - 30 June: feature freeze and integration testing.
 - 1 July: bug fixing and demo rehearsal.
- **Key Milestones and Deliverables:** Complete the core Publisher Tool foundation: login/RBAC, API specification upload, metadata capture, basic specification validation, validation feedback display, and draft/resubmission support. Prepare the Week 5 progressive demo and Sprint 1 retrospective.

2.4.2. Sprint 2

- **Calendar Dates:** 2 July 2026 - 23 July 2026
- **Main Focus:** Major validation features and repository integration. This period includes the flexibility week, so active development is planned around team availability.
- **User Stories / Backlog Focus:** Backlog items to be selected after Sprint 1 review based on team velocity. Stories will be drawn from the product backlog and prioritised according to dependencies and remaining capacity.
- **Internal Deadlines :**
 - 4 July: Sprint 2 backlog selection confirmed after Sprint 1 review.
 - 9 July: domain/security validation rule design completed.
 - 15 July: validation modules implementation checkpoint.
 - 17 July: repository publication prototype completed.
 - 20 July: feature freeze and integration testing.
 - 22 July: demo rehearsal and retrospective preparation.
- **Key Milestones and Deliverables:** Extend the validation pipeline with domain compliance and security metadata validation. Integrate successfully validated API records with the Central API Repository. Prepare API listing/status overview and deliver an integrated prototype for the Week 8 progressive demo.

2.4.3. Sprint 3

- **Calendar Dates:** 23 July 2026 - 6 August 2026
- **Main Focus:** Final integration, refinement, testing, and project completion.
- **User Stories / Backlog Focus:** Remaining backlog items to be finalised after Sprint 2 review, prioritised based on team velocity and outstanding deliverables.
- **Internal Deadlines :**
 - 25 July: Sprint 3 scope confirmed after Sprint 2 review.
 - 29 July: update/withdrawal workflow and schema mapping draft completed.

- 31 July: unified UI integration completed.
- 3 August: final feature freeze and end-to-end testing.
- 5 August: final demo rehearsal, report check, and codebase quality check.
- **Key Milestones and Deliverables:** Complete remaining API management and ecosystem integration features. Finalise update/withdrawal workflows, unified ecosystem UI integration, schema mapping support, bug fixing, final demo preparation, final report writing, and software quality/codebase submission.

Planning note: Sprint 1 contains 29 story points out of a total of 71 story points. Although this is slightly more than one third of the total project estimate, these stories were selected because they establish the core foundation of the API Publisher workflow. Sprint 2 and Sprint 3 backlog items will be finalised progressively after sprint reviews, following the Scrum principle that future backlog items should not be pre-assigned before team velocity is known.

Internal deadline rationale: Each sprint includes checkpoints before the sprint endpoint so that the team has time for integration testing, bug fixing, demo rehearsal, and retrospective preparation. This reduces the risk of presenting incomplete or untested functionality at the progressive demos and final demonstration.

3. Technical Design

3.1. System Architecture Diagram

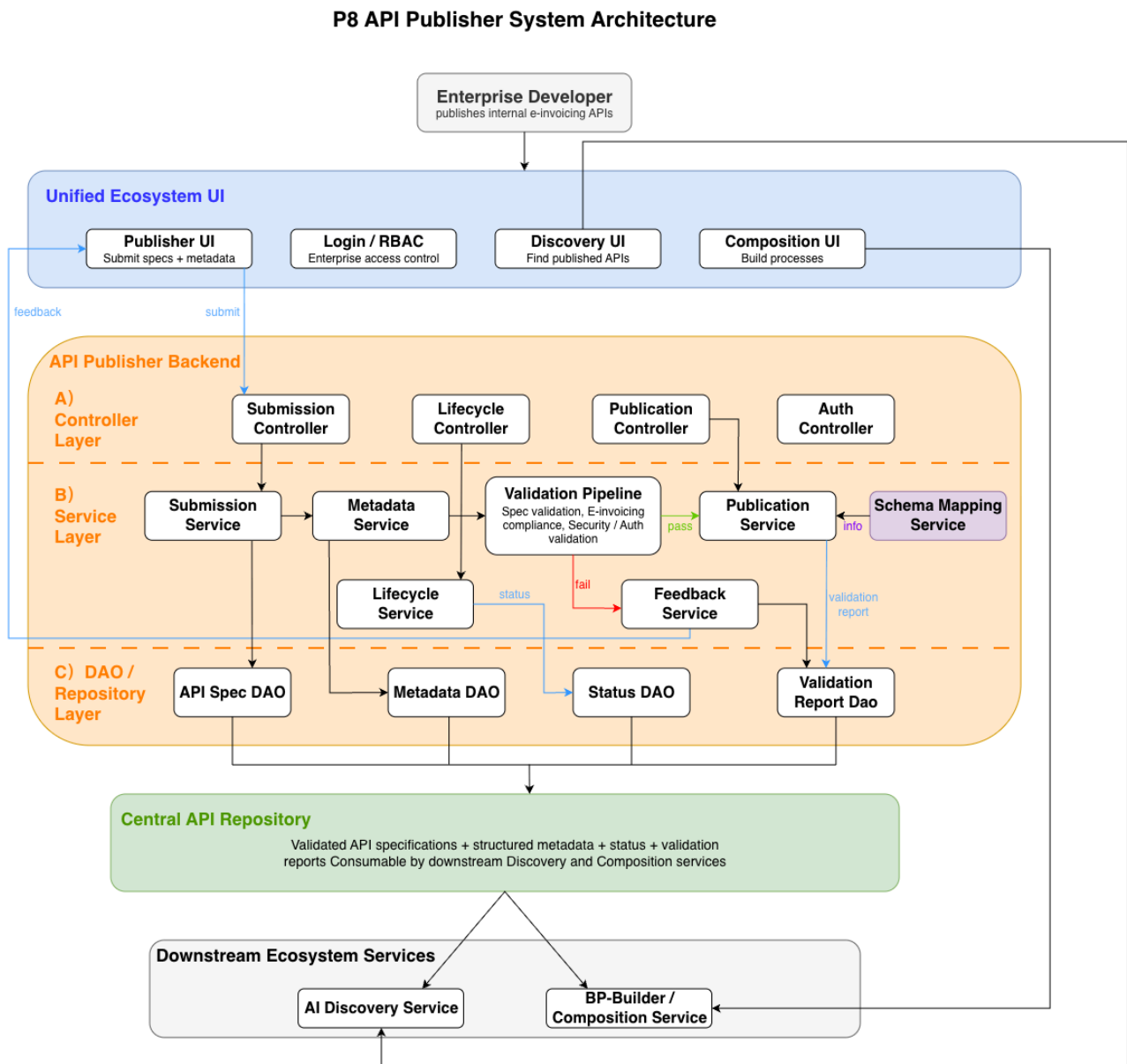


Figure 3: P8 API Publisher System Architecture

Architecture is broken into four layers. **Unified Ecosystem UI** includes Publisher UI, Login/RBAC, Discovery UI, and Composition UI, which serve one purpose - integrating an API lifecycle within one solution so users can easily access a unified set of resources to discover and access APIs. Thus, the Publisher UI submits API spec and metadata to the back end; return validation feedback/status through same channel.

The **API Publisher Back-end** is additionally divided into three layers of functionality (each layer being a specific area of concern within the API Publisher). Layer 1 - The **Controller Layer** consists of different controllers that serve as a point of entry into the API Publisher ; each controller handles process routing of request to the proper service layer. Layer 2 - The **Service Layer** contains all of the business logic for the API Publisher. For example, the Submission Service receives an API Spec to

submit , hands off API Metadata to the Metadata Service for capturing both submission information and the individual transaction record . Once validated through all three processes, the Publication Service saves the API Spec to the repository but will provide a structured error report if any of the three validation paths fail. Additionally, the Lifecycle Service handles management of a submitted and draft status; while the Schema Mapping Service will provide the necessary metadata to the Publication Service that will facilitate proper field mapping. Layer 3 - The **DAO Layer** consists of the API Spec DAO, the Metadata DAO, the Status DAO, and the Validation Report DAO, which provide a consistent view of the actual data from each of these different perspectives; the result of which is that these various DAOs allow data to be retrieved from the proper persistent storage source.

The **Central API Repository** stores validated specifications, metadata, status, and validation reports in a format consumable by the downstream **AI Discovery Service** and **BP-Builder / Composition Service**, completing the ecosystem integration.

3.2. Interface storyboarding

The API Publisher tool's interface is designed to match the whole e-invoicing architecture visually. In this section, we will develop five non-detailed logical storyboard including significant operational use-case/scenarios that have a direct structural relationship among them through their associated user story and Functional Requirements (FR). Additionally, these same FRs will provide a formalised traceability matrix during the design step. The visual overall layout will be that of one functional cohesive visually related component hierarchy and will be constructed from an enterprise-level semantic colour palette as used throughout to maintain continuity in readability and meaning for all users throughout their navigation through the system.

3.2.1. Scenario 1: Unified Enterprise Authentication Gate and Publisher Dashboard

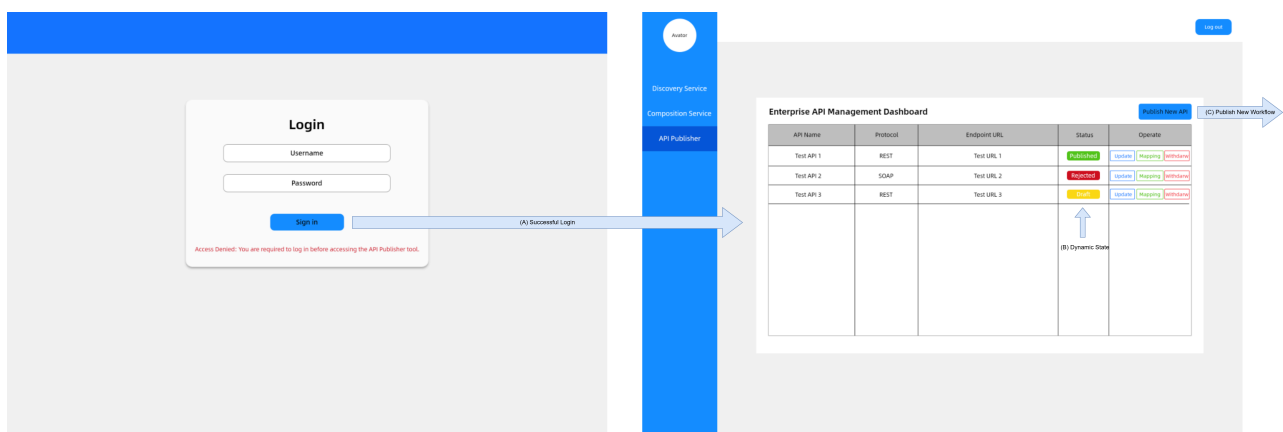


Figure 4: Storyboarding Scene 1

- **Covered Jira User Stories:** FR-1 (Publisher Authentication and Authorization), FR-10 (Listing of Published APIs), FR-12 (Integration with Unified UI).
- **Interface Layout & Basic Colour Scheme:** The screen uses a light-grey canvas background throughout. The left side shows a centred login card with a white background. After authentication,

the right side displays the publisher dashboard: a blue vertical navigation sidebar occupies the left edge, and the main workspace uses a white card containing the API data grid.

- **Component Hierarchy:**

- **Left Navigation Sidebar:** [User Avatar] → [Discovery Service] → [Composition Service] → [API Publisher (active, highlighted in blue)].
- **Dashboard Header:** [Enterprise API Management Dashboard title] → [Publish New API button] → [Publish New Workflow button].
- **API Data Grid:** [API Name] | [Protocol] | [Endpoint URL] | [Status] | [Operate: Update / Mapping / Withdraw].

- **Dynamic Navigation & Interaction Flows:**

- **Flow (A) — Authentication Interception:** Unauthenticated users are shown the login screen with an access-denied message. After submitting valid credentials via [Sign In], the system redirects the user to the publisher dashboard (FR-1).
- **Flow (B) — Dynamic Status Rendering:** Each API record in the data grid is labelled with a colour-coded status badge: Published (green), Rejected (red), and Draft (yellow), reflecting its current lifecycle state in the central repository (FR-10).
- **Flow (C) — Operational Routing:** Clicking [Update] or [Withdraw] on a published row routes the user to the corresponding lifecycle workflow (FR-11). Clicking [Publish New API] or [Publish New Workflow] navigates the user to the multi-step submission wizard in Scenario 2.

3.2.2. Scenario 2: Guided Multi-Protocol API Upload and Metadata Capture Form

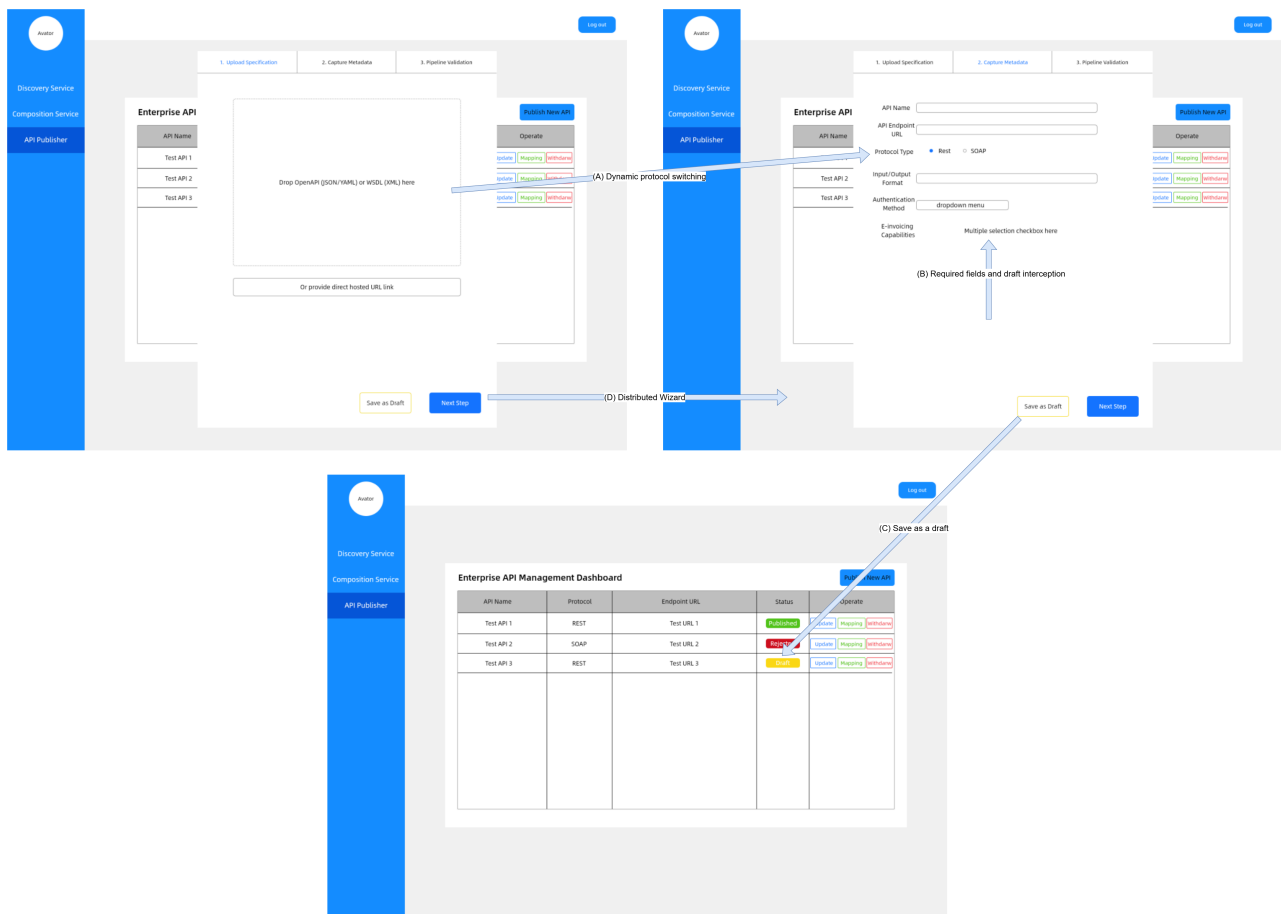


Figure 5: Storyboarding Scene 2

- **Covered Jira User Stories:** FR-2 (API Specification Upload), FR-3 (Metadata Capture Form), FR-9 (Draft and Resubmission Support).
- **Interface Layout & Basic Colour Scheme:** This scenario spans two wizard steps displayed within the same white-background card layout, sharing the blue left navigation sidebar from Scenario 1. A linear stepper bar at the top tracks progress across all three stages.
- **Component Hierarchy:**
 - **Workflow Stepper:** [1. Upload Specification (active)] → [2. Capture Metadata] → [3. Pipeline Validation].
 - **Step 1 — Specification Upload Panel:** [Drag-and-drop zone: “Drop OpenAPI (JSON/YAML) or WSDL (XML) here”] | [URL input: “Or provide direct hosted URL link”].
 - **Step 2 — Metadata Capture Form:** [API Name] → [API Endpoint] → [Protocol Type: REST / SOAP radio group] → [Input/Output Format] → [Authentication Method dropdown] → [E-invoicing Capabilities multi-select checkboxes].
 - **Action Bar (both steps):** [Save as Draft] | [Next Step].
- **Dynamic Navigation & Interaction Flows:**

- **Flow (A) — Dynamic Protocol Switching:** Uploading a `.wsdl` file or a WSDL-format URL automatically switches the Protocol Type radio selection to `SOAP`. Uploading an OpenAPI (JSON/YAML) file switches it to `REST` (FR-2). Attaching an unsupported file type highlights the upload zone with a red border and displays an error message.
- **Flow (B) — Required Fields and Draft Interception:** All form fields are validated before step navigation. Clicking `[Next Step]` with incomplete mandatory fields blocks the transition, highlights the empty fields in red, and displays a validation warning (FR-3).
- **Flow (C) — Save as Draft:** Clicking `[Save as Draft]` on either step saves the current inputs and redirects the user back to the dashboard (Scenario 1), where the saved API appears in the data grid with a yellow `Draft` status badge (FR-9).
- **Flow (D) — Distributed Wizard Navigation:** Completing Step 1 and clicking `[Next Step]` advances the stepper to Step 2 (Capture Metadata), where the metadata form is presented within the same card layout without a full page reload.

3.2.3. Scenario 3: Asynchronous Multi-Stage Validation Pipeline and Failure Console

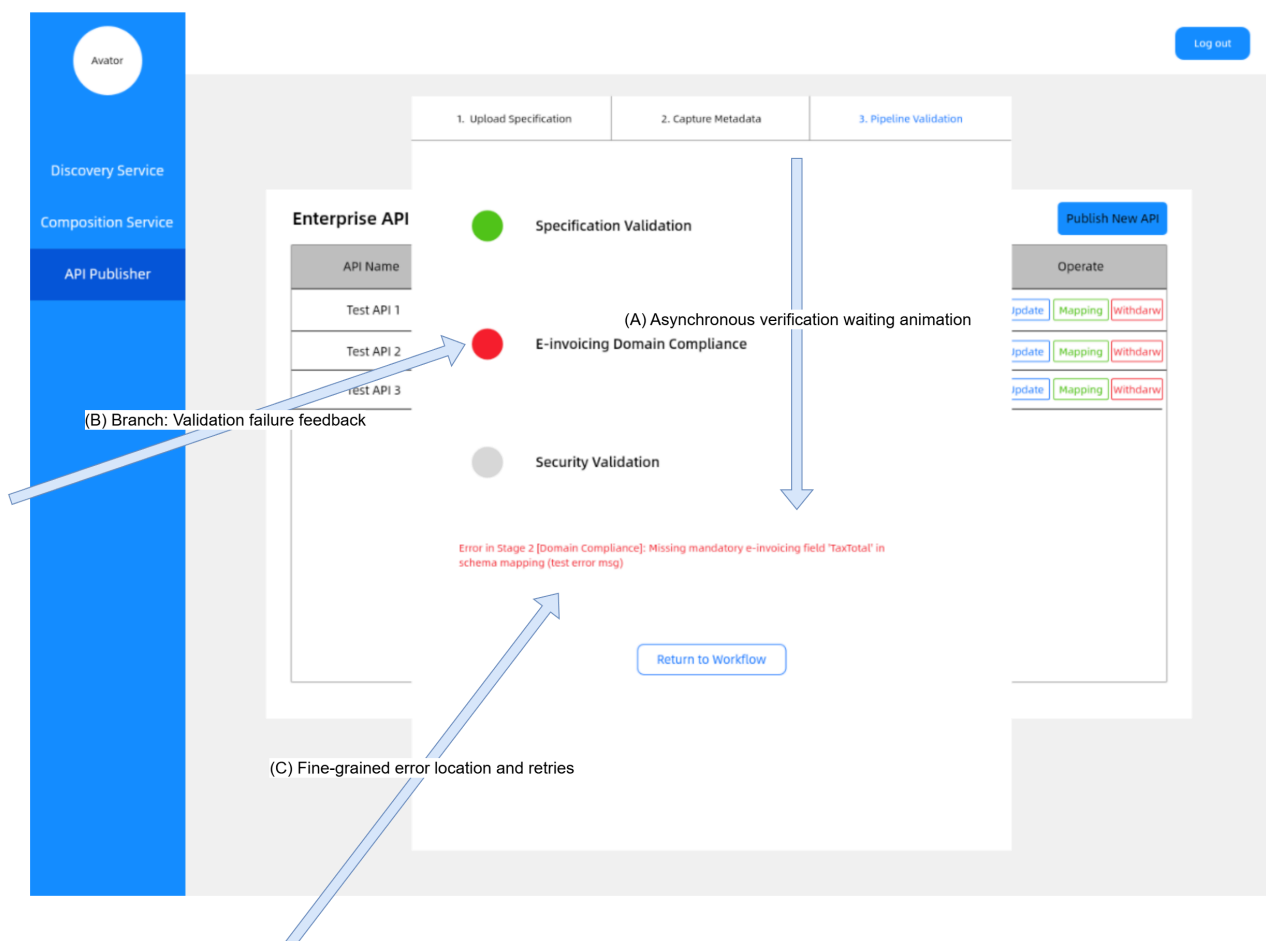


Figure 6: Storyboarding Scene 3

- **Covered Jira User Stories:** FR-4 (Specification Validation), FR-5 (E-Invoicing Domain Compliance Validation), FR-6 (Security and Authentication Metadata Validation), FR-7 (Validation Feedback).

- **Interface Layout & Basic Colour Scheme:** This view is reached at Step 3 of the wizard (Pipeline Validation active in the stepper). The main card displays three vertically stacked validation nodes, each represented by a colour-coded circle. Validation errors are shown as inline red text below the pipeline nodes, followed by a `[Return to Workflow]` button.
- **Component Hierarchy:**
 - **Validation Pipeline Nodes:**
 - Node 1: Specification Validation
 - Node 2: E-invoicing Domain Compliance
 - Node 3: Security Validation
 - **Inline Error Display:** `[Red error message text]` → `[Return to Workflow]` button.
- **Dynamic Navigation & Interaction Flows:**
 - **Flow (A) — Asynchronous Verification Animation:** After the user submits from Scenario 2, each node shows a loading state while its backend check runs. On completion, the node circle updates to green (passed) (FR-4).
 - **Flow (B) — Validation Failure Feedback:** If domain compliance fails, Node 1 remains green, Node 2 turns red (failed), and Node 3 turns grey (bypassed). The failure is reflected in the dashboard API list on the left (FR-5 , FR-6).
 - **Flow (C) — Fine-grained Error Location and Retries:** The failed stage displays an inline error message below the pipeline: *“Error in Stage 2 [Domain Compliance]: Missing mandatory e-invoicing field ‘TaxTotal’ in schema mapping”* (FR-7). Clicking `[Return to Workflow]` navigates the user back to the Scenario 2 form with all previously entered data preserved, allowing correction and resubmission (FR-9).

3.2.4. Scenario 4: Visual Interactive API Schema Mapping Canvas

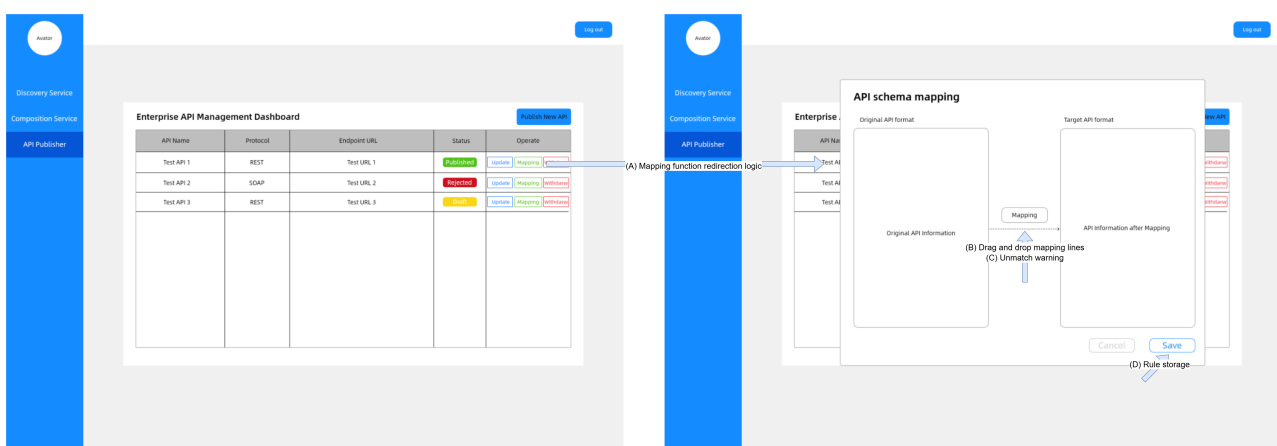


Figure 7: Storyboarding Scene 4

- **Covered Jira User Stories:** FR-13 (API Schema Mapping).

- **Interface Layout & Basic Colour Scheme:** The schema mapping screen is presented as a modal overlay triggered from the dashboard. It displays two side-by-side panels on a white background, with a central `[Mapping]` button between them and `[Cancel]` / `[Save]` buttons at the bottom.
- **Component Hierarchy:**
 - **Left Panel:** Original API format (source schema fields) and Original API information.
 - **Right Panel:** Target API format (target schema fields) and API information after Mapping.
 - **Centre:** `[Mapping]` button.
 - **Footer Controls:** `[Cancel]` | `[Save]`.
- **Dynamic Navigation & Interaction Flows:**
 - **Flow (A) — Mapping Function Redirection:** Clicking the `[Mapping]` button on any row in the dashboard opens the schema mapping modal for that API, loading its source schema fields in the left panel (`FR-13`).
 - **Flow (B) — Drag and Drop Mapping Lines:** The user draws connections between source fields (left) and target fields (right) by dragging. Successfully matched fields are linked with a visible line, and the right panel updates to reflect the mapped output (`FR-13`).
 - **Flow (C) — Unmatch Warning:** If a source field cannot be matched to a compatible target field (e.g. due to a type mismatch), the system displays a warning indicating that the mapping cannot be completed for that field (`FR-13`).
 - **Flow (D) — Rule Storage:** Clicking `[Save]` stores the completed field mappings for use by downstream Discovery and Composition services. Clicking `[Cancel]` closes the modal without saving changes.

3.2.5. Scenario 5: Successful Publication Registry Entry and Cross-Service Lifecycle Controls

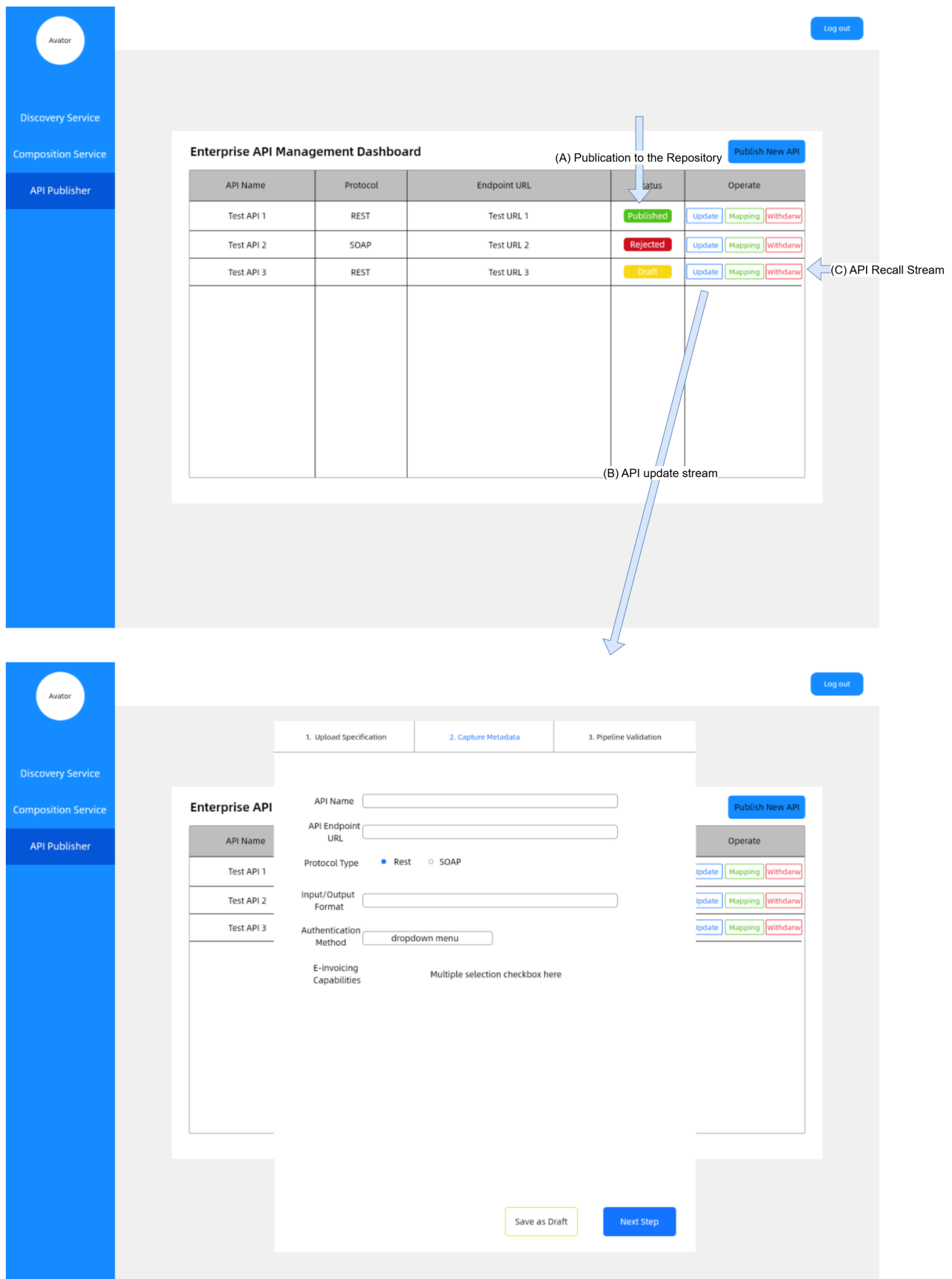


Figure 8: Storyboarding Scene 5

- **Covered Jira User Stories:** FR-8 (Publication to the Repository), FR-11 (Update and Withdrawal of Published APIs).
- **Interface Layout & Basic Colour Scheme:** This scenario shows two panels. The top panel is the dashboard from Scenario 1, with the newly published API appearing in the data grid. The bottom panel shows the Capture Metadata form (Scenario 2, Step 2), reached via the Update flow.
- **Component Hierarchy:**
 - **Dashboard Operate Column:** [Update] | [Mapping] | [Withdraw] buttons on each API row.
 - **Status Badges:** Published (green), Rejected (red), Draft (yellow).
- **Dynamic Navigation & Interaction Flows:**
 - **Flow (A) — Publication to the Repository:** After completing the validation pipeline, the API is published to the Central Repository and appears in the dashboard with a green Published status badge (FR-8).
 - **Flow (B) — API Update Stream:** Clicking [Update] on a published API row routes the user back to the Capture Metadata form (Step 2 of the Scenario 2 wizard) with existing data pre-filled, allowing edits before resubmission through the validation pipeline (FR-11).
 - **Flow (C) — API Recall Stream:** Clicking [Withdraw] on a published API initiates the withdrawal process, removing the API from active discovery and composition availability and updating its status in the dashboard (FR-11).

3.2.6. Traceability and Verification Matrix

The following matrix maps each storyboard screen (SCR-01 through SCR-05) to its supported Jira functional requirements and key UI components, confirming complete coverage of all major system functionalities.

Screen ID	Screen Name	Jira Issue Keys	Key UI Components
SCR-01	Unified Portal Dashboard	FR-1, FR-10, FR-12	Left nav sidebar, status badges, Update / Mapping / Withdraw controls
SCR-02	Guided Submission Form	FR-2, FR-3, FR-9	Wizard stepper, drag-and-drop upload zone, metadata form, Save as Draft
SCR-03	Validation Pipeline Console	FR-4, FR-5, FR-6, FR-7	Colour-coded pipeline nodes, inline error message, Return to Workflow button
SCR-04	Schema Mapping Canvas	FR-13	Source / target schema panels, drag-and-drop mapping lines, unmatched warning
SCR-05	Publication & Lifecycle	FR-8, FR-11	Published status badge, Update route to metadata form, Withdraw action

Table 1: Storyboard-to-requirement traceability matrix.

3.3. Design Justifications

This section justifies the major design decisions for the proposed API Publisher Tool.² The overall problem is broken into smaller subproblems so that each part of the solution can be evaluated against alternatives. These design decisions directly respond to the problem definition that enterprise e-invoicing APIs are fragmented across systems and teams, inconsistently documented, difficult to discover, and often lack standardised machine-readable information about specifications, authentication, supported invoice standards, schemas, and mapping requirements. The proposed design follows the system architecture, including the Unified Ecosystem UI, controller layer, service layer, DAO/repository layer, Central API Repository, and downstream Discovery and Composition services. The design has also been aligned with the interface storyboards: SCR-01 supports the unified dashboard and authentication flow, SCR-02 supports guided submission and draft saving, SCR-03 supports staged validation feedback, SCR-04 supports schema mapping, and SCR-05 supports publication and lifecycle maintenance.

Subproblem	Alternative Solution(s)	Chosen Solution	Reason for Selection	User Impact	Limitation / Mitigation	Novelty / Link to Existing Systems
Access control and authentication for enterprise publishers	Allow all logged-in users to access publishing functions; or manage permissions manually outside the system.	Use Login/RBAC with an Auth Controller so that only authorised enterprise users can create, edit, delete, update, or withdraw API submissions. This is reflected in SCR-01, where users are redirected from login into the unified publisher dashboard only after authentication.	Role-based access control is more suitable because API publication affects the quality and reliability of the central repository. It separates ordinary access from restricted publishing actions and reduces governance risk. SCR-01 reinforces this decision by showing the authentication gate before dashboard access.	Enterprise publishers receive access only to functions they are permitted to use, while unauthorised users receive clear denial messages instead of silently failing.	RBAC adds setup complexity because user roles must be maintained. This is mitigated by keeping roles simple in the initial release and applying role checks at controller level for restricted actions.	This extends general API portal access control by linking authorisation directly to API lifecycle actions such as submission, update, and withdrawal.
Complex API submission process requiring both technical specifications	Require publishers to upload only a raw OpenAPI/WSDL file; or	Use a structured submission workflow with specification upload,	A raw specification alone is not enough for validation, discov-	Publishers are guided through the required information instead of	Longer forms may increase user effort. This is mitigated by	The design adopts the design-first clarity seen in API documentation

²Since the tables in the following sections contain a large amount of data, and to ensure a visually appealing layout, they will be displayed in landscape A4 format.

Subproblem	Alternative Solution(s)	Chosen Solution	Reason for Selection	User Impact	Limitation / Mitigation	Novelty / Link to Existing Systems
tions and business metadata	collect metadata later through manual review.	URL input, metadata capture form, capability selection, Submission Controller, Submission Service, Metadata Service, API Spec DAO, and Metadata DAO. SCR-02 supports this through a guided multi-step submission form.	ery, and composition. The problem definition identifies that enterprise APIs are often documented in inconsistent ways and lack machine-readable representations of specifications, authentication, supported invoice standards, schemas, and mapping requirements. SCR-02 reduces this risk by making upload, metadata capture, mandatory field checks, and draft saving visible in one guided workflow.	guessing what must be provided. Mandatory fields reduce incomplete submissions and reduce reliance on informal “tribal knowledge”.	clearly marking mandatory fields, using a step-per layout, and supporting draft saving so publishers can complete submissions gradually.	platforms while tailoring metadata capture to the e-invoicing ecosystem and downstream service needs.
Maintaining validation quality across API structure, domain compliance, and security metadata	Use one large validation function for all checks; or rely on manual review by project members or administrators.	Use a modular Validation Pipeline with separate specification validation, e-invoicing domain compliance validation, security/authentication metadata validation, Feedback Service, and Validation Report DAO. SCR-03 visualises these stages as separate validation nodes.	Modular validation makes each validation concern easier to maintain, test, and extend. It also addresses the identified problem that many existing e-invoicing APIs have unclear compliance status, inconsistent input/output formats, and incomplete authentication information. SCR-03 supports this decision by showing	Publishers receive clearer validation outcomes, including which stages passed or failed and why, making correction and resubmission more efficient.	The first implementation may not cover every possible e-invoicing rule. This is mitigated by designing the pipeline so additional validation rules can be added later without rewriting the full workflow.	The solution combines specification validation, domain-specific compliance checking, and repository governance within one e-invoicing publication workflow.

Subproblem	Alternative Solution(s)	Chosen Solution	Reason for Selection	User Impact	Limitation / Mitigation	Novelty / Link to Existing Systems
Handling failed submissions without forcing publishers to restart from the beginning	Reject invalid submissions permanently and require publishers to create a new submission each time; or ask users to manually copy previous information.	Use Draft Management, Lifecycle Controller, Lifecycle Service, Status DAO, and a resubmission workflow with states such as draft, rejected, and resubmitted. SCR-02 shows draft saving, while SCR-03 returns users to the workflow after validation failure.	specification validation, domain compliance, and security validation as separate pass/fail stages. API publication is iterative. Draft and resubmission support reduces duplicated work and allows invalid submissions to be corrected using the existing stored specification and metadata. The storyboard flow supports this by preserving user input after a validation error.	Publishers can save incomplete work, reopen drafts, edit failed submissions, and resubmit without re-entering all details.	Maintaining submission states increases backend complexity. This is mitigated by using a dedicated lifecycle service and status DAO so state changes are controlled consistently.	This improves the developer experience beyond a simple validation portal by treating publication as a managed lifecycle rather than a one-time upload.
Ensuring only valid APIs are published to the Central API Repository	Store every submitted API directly in the repository and mark invalid APIs later; or use a separate manual approval process after upload.	Use a Publication Controller and Publication Service that publish to the Central API Repository only after required validation stages are completed successfully. SCR-05 supports this by showing successful publication feedback and dashboard status changes.	Publishing only validated APIs protects repository quality and prevents downstream Discovery and Composition services from consuming incomplete or non-compliant records. This is important because the current problem is not simply a lack of APIs, but the lack of a controlled and standardised publishing process for APIs across different	Publishers receive clear success or failure messages. Downstream users can trust that published API records have passed the required checks.	Strict publication gates may delay publication when validation rules are too restrictive. This is mitigated through detailed error reports and resubmission support.	This design integrates validation and repository governance in one workflow, addressing the gap between design-time tools and runtime registry systems.

Subproblem	Alternative Solution(s)	Chosen Solution	Reason for Selection	User Impact	Limitation / Mitigation	Novelty / Link to Existing Systems
Managing published APIs after publication when APIs become outdated or incorrect	Treat published APIs as permanent records; or allow direct editing in the repository without revalidation.	Use Lifecycle Controller, Lifecycle Service, Publication Service, and Central API Repository to support update and withdrawal, with revalidation before republishing updated APIs. SCR-05 shows update and withdrawal controls and confirmation feedback.	enterprise teams and systems. Published APIs may change over time, but uncontrolled edits could break downstream services. Revalidation before republication balances flexibility with repository reliability. SCR-05 supports this lifecycle decision by routing updated APIs back through validation and warning users before withdrawal.	Publishers can correct outdated or incorrect APIs, while withdrawn APIs are removed from discovery and composition availability.	Revalidation may slow down updates. This is mitigated by reusing the same validation pipeline and providing confirmation messages when updates or withdrawals are completed.	This adds lifecycle governance beyond static documentation by ensuring published APIs can evolve safely.
Supporting schema interoperability for downstream Discovery and Composition services	Leave schema interpretation entirely to downstream services; or manually document field relationships without storing structured mappings.	Use a Schema Mapping Service, Schema Field Matcher, Mapping Storage, Central API Repository, AI Discovery Service interface, and BP-Builder/Composition Service interface. SCR-04 supports this through a visual mapping canvas.	Centralised schema mapping reduces duplicated work and helps align API inputs and outputs across different providers and formats, including JSON payloads, UBL XML, and proprietary vendor formats. This directly responds to the problem that current e-invoicing APIs often lack mapping or validation between different schemas. SCR-04	Downstream services can use stored schema mappings to discover, compare, and compose APIs more effectively.	Automatic field matching may produce incomplete or uncertain mappings. This is mitigated by notifying users when fields cannot be matched and by storing mapping information for review and reuse.	This builds beyond conventional API documentation by supporting machine-usable schema alignment for ecosystem-level discovery and composition.

Subproblem	Alternative Solution(s)	Chosen Solution	Reason for Selection	User Impact	Limitation / Mitigation	Novelty / Link to Existing Systems
Providing a consistent interface across publishing, discovery, and composition services	Build the API Publisher Tool as a standalone web application; or use separate interfaces for publishing, discovery, and composition.	Integrate Publisher UI, Discovery UI, Composition UI, Login/RBAC, and navigation within the Unified Ecosystem UI. SCR-01 demonstrates this by placing the Publisher Dashboard inside the same portal layout as Discovery and Composition services.	makes this design visible through source and target schema panels, mapping controls, and mismatch warnings. A unified interface is more suitable for an API ecosystem because users should move between publishing, discovery, and composition services without switching systems or learning different navigation patterns. SCR-01 supports this decision through a left-side navigation structure and shared dashboard layout.	Enterprise users gain a more consistent workflow and can understand where the Publisher section fits within the wider ecosystem.	Unified UI integration requires coordination with other service interfaces. This is mitigated by using shared navigation and consistent screen structure while allowing each service to remain modular.	This extends the project beyond a standalone publisher by positioning it as one integrated part of a broader e-invoicing API ecosystem.

Table 2: Design justification by subproblem, alternatives, chosen solution, user impact, mitigation, novelty, and storyboard alignment.

Summary: The main goal of this design is to solve the major issues facing the project by breaking down the various areas of responsibility — access control, API submission, validating API submissions, managing the lifecycle of each API, publishing an API, mapping API schemas to the ecosystem, etc. — into smaller, more manageable pieces. As such, the design also addresses the primary “background problem”: moving enterprise organizations away from disjointed documentation and reliance on “tribal knowledge”, and toward a controlled, standard process for creating machine-readable API documentation.

In contrast to other systems that document or store API specifications, the proposed design does not just address the above issues; rather, it creates a complete e-invoicing API specification workflow through uploading API specifications, capturing metadata for each specification, validating API submissions through staged results and feedback, publishing validated API specifications into an API repository, using the published API specifications to

support schema mapping for downstream applications, and designing the entire project from the standpoint of the API consumer, with minimal disruption to current business processes.

While this design will initially limit some of the advanced features of the system — such as domain validation and schema mapping — to a limited set of rules, these limitations will be alleviated over time by developing modular services and creating storyboard-supported interaction flows that enable the modular services to be expanded as the project evolves.

3.4. Functionality Mapping

The table below maps the API Publisher Tool to the relevant Jira user stories, interface storyboard designs, and system architecture components that support its functionality. The architecture components are based on the P8 API Publisher System Architecture, including the Unified Ecosystem UI, API Publisher Backend Controller Layer, Service Layer, DAO/Repository Layer, Central API Repository, and downstream Discovery and Composition services. The additional interface storyboard traceability links each function to the corresponding interface scenarios, SCR-01 to SCR-05, demonstrating that the planned functionality is aligned with the user requirements, UI designs, and underlying technical architecture. These functionalities respond to the identified problems of fragmented e-invoicing APIs, inconsistent documentation, unclear authentication requirements, limited machine-readable API representations, and weak schema mapping support across enterprise systems.

Functionality	Related Story / Jira Key	User Story- board	Architecture Component	Justification
Publisher authentication and authorization	H19BBREAD-1 / FR-1 - Publisher authentication and authorization	SCR-01 - Unified Portal Dashboard	Unified Ecosystem UI; Login / RBAC; Auth Controller	This functionality is satisfied by FR-1 because the user story requires enterprise publishers to authenticate before accessing the API Publisher tool. The storyboard supports this through the login gate and authenticated dashboard entry, while the architecture supports enforcement through Login / RBAC and the Auth Controller. This prevents unauthorised users from creating, modifying, or deleting API submissions and provides a clear access-denied message when permissions are insufficient.
API specification upload	H19BBREAD-2 / FR-2 - API specification upload	SCR-02 - Guided Submission Form	Publisher UI; Submission Controller; Submission Service; API Spec DAO	This functionality is satisfied by FR-2 because the user story allows publishers to upload OpenAPI/Swagger or WSDL specifications, or provide a hosted specification URL. SCR-02 shows the upload container, hosted URL input, and protocol-aware submission wizard. The architecture supports this through the Publisher UI and Submission Controller, while the Submission Service coordinates specification storage through the API Spec DAO.

Functionality	Related Story / Jira Key	User	Related board	Story-	Architecture Component	Justification
API metadata collection	H19BBREAD-3 / FR-3 - Metadata capture form		SCR-02 - Submission Form	Guided	Publisher UI; Submission Controller; Metadata Service; Metadata DAO	This functionality is satisfied by FR-3 because the user story requires a structured metadata capture process for API name, endpoint URL, protocol type, input/output formats, authentication method, API description, and e-invoicing capabilities. SCR-02 visualises these fields in the metadata step of the guided wizard. The Metadata Service and Metadata DAO store this information as part of the submission record, supporting validation, discovery, and downstream service use.
API specification validation	H19BBREAD-4 / FR-4 - Specification validation		SCR-03 - Validation Pipeline Console	Validation	Validation Pipeline; Feedback Service; Validation Report DAO	This functionality is satisfied by FR-4 because the user story requires uploaded OpenAPI/ Swagger and WSDL documents to be validated before publication. SCR-03 shows the specification validation stage as part of the multi-stage validation pipeline. The Validation Pipeline checks syntax, required elements, element types, and structural consistency. Failed validations are passed to the Feedback Service and stored through the Validation Report DAO so publishers can correct issues before resubmission.
E-invoicing domain compliance validation	H19BBREAD-7 / FR-5 - E-invoicing domain compliance validation		SCR-03 - Validation Pipeline Console	Validation	Validation Pipeline; Feedback Service; Validation Report DAO; Central API Repository	This functionality is satisfied by FR-5 because the user story requires API submissions to be checked against recognised e-invoicing domain rules before acceptance into the repository. SCR-03 visualises domain compliance as a separate pipeline stage, including failure states and targeted feedback. The Validation Pipeline evaluates API inputs, outputs, and operations for domain compliance. Non-compliant APIs are rejected before publication and domain-focused error reports are generated through the feedback and reporting components.
Security and authentication metadata validation	H19BBREAD-8 / FR-6 - Security and authentication metadata validation		SCR-03 - Validation Pipeline Console	Validation	Validation Pipeline; Auth Controller; Feedback Service; Validation Report DAO	This functionality is satisfied by FR-6 because the user story requires security and authentication metadata to be complete, acceptable, and internally consistent with the submitted API specification. SCR-03 includes security validation as a distinct pipeline stage. The Validation Pipeline checks declared mechanisms such as OAuth 2.0, API Key, Basic Authentication, and mTLS, while the Auth Controller supports authentication-related governance. Failed submissions receive clear security-focused feedback before resubmission.
Validation feedback and error reporting	H19BBREAD-5 / FR-7 - Validation feedback		SCR-03 - Validation Pipeline Console	Validation	Feedback Service; Validation Report DAO; Publisher UI	This functionality is satisfied by FR-7 because the user story requires clear and structured validation feedback after an API is submitted. SCR-03 shows pass/fail pipeline stages, an error console, and a return-to-workflow path. The Feedback Service and Validation Report DAO record which stages passed or failed, explain the reason for each failure,

Functionality	Related Story / Jira Key	User	Related board	Story-	Architecture Component	Justification
API publication to central repository	H19BBREAD-9 / FR-8 - Publication to the Repository		SCR-05 - Success & Maintenance Flow		Publication Controller; Publication Service; Central API Repository; AI Discovery Service; BP-Builder / Composition Service	and highlight issues requiring correction. The Publisher UI displays this information so publishers can return to the submission workflow and resubmit corrected APIs. This functionality is satisfied by FR-8 because the user story requires successfully validated APIs to be published to the Central API Repository. SCR-05 shows the successful publication state and confirms that the API is available for downstream services. The Publication Controller and Publication Service ensure publication only occurs after all required validation stages pass. The resulting API specification and metadata are stored in the repository in a format usable by downstream AI Discovery and BP-Builder / Composition services.
Draft saving and resubmission workflow	H19BBREAD-6 / FR-9 - Draft and resubmission support		SCR-02 - Guided Submission Form; SCR-03 - Validation Pipeline Console		Lifecycle Controller; Lifecycle Service; Status DAO; Submission Service; Publisher UI	This functionality is satisfied by FR-9 because the user story allows incomplete submissions to be saved as drafts and failed submissions to be corrected and resubmitted. SCR-02 includes the Save as Draft action, while SCR-03 shows return-to-workflow behaviour after validation failure. The Lifecycle Controller and Lifecycle Service manage submission states such as draft, rejected, and resubmitted, while the Status DAO records the current state. This prevents publishers from re-entering information and supports an iterative correction workflow.
Published API listing and status overview	H19BBREAD-10 / FR-10 - Listing of published APIs		SCR-01 - Unified Portal Dashboard		Publisher UI; Central API Repository; Status DAO; Discovery UI	This functionality is satisfied by FR-10 because the user story requires publishers to view APIs registered by their enterprise and monitor their current status. SCR-01 shows the dashboard table with API name, protocol, endpoint URL, status tags, and operation controls. The Publisher UI retrieves API details and statuses from the Central API Repository and Status DAO. This gives publishers a clear overview of published, rejected, and draft APIs while restricting visibility to APIs belonging to the registered enterprise.
API update and withdrawal management	H19BBREAD-11 / FR-11 - Update and withdrawal of published APIs		SCR-05 - Success & Maintenance Flow		Lifecycle Controller; Lifecycle Service; Publication Service; Validation Pipeline; Central API Repository	This functionality is satisfied by FR-11 because the user story allows publishers to update outdated API metadata, submit revised specifications, or withdraw published APIs. SCR-05 shows update and withdrawal controls, revalidation routing, and safety confirmation for withdrawal. Updated APIs are routed through the Validation Pipeline before republication, maintaining repository quality. Withdrawn APIs are removed from availability for Discovery and Composition services, while lifecycle and publication components manage status changes and confirmation messages.

Functionality	Related Story / Jira Key	User Story-	Related board	Architecture Component	Justification
Unified ecosystem UI integration	H19BBREAD-12 / FR-12 - Integration with unified UI	SCR-01 - Unified Portal Dashboard		Unified Ecosystem UI; Publisher UI; Discovery UI; Composition UI	This functionality is satisfied by FR-12 because the user story requires the API Publisher tool to be accessible through the unified ecosystem interface. SCR-01 places Discovery Service, Composition Service, and API Publisher in a shared navigation structure. The architecture explicitly places the Publisher UI, Login / RBAC, Discovery UI, and Composition UI inside the Unified Ecosystem UI. This supports consistent navigation and interaction across publishing, discovery, and composition services without forcing users to switch between separate systems.
API schema mapping for downstream services	H19BBREAD-13 / FR-13 - API schema mapping	SCR-04 - Schema Mapping Canvas		Schema Mapping Service; Publication Service; Central API Repository; AI Discovery Service; BP-Builder / Composition Service	This functionality is satisfied by FR-13 because the user story requires schema mapping between APIs and additional data formats so that inputs and outputs can be aligned for downstream use. SCR-04 shows the schema mapping canvas with source and target schema areas, mapping controls, unmatched-field warnings, and save behaviour. The Schema Mapping Service provides field matching and mapping support, while mapped information is stored through the publication and repository flow. This supports downstream AI Discovery and BP-Builder / Composition services by making API schemas easier to align, reuse, and extend, especially where different providers use JSON payloads, UBL XML, or proprietary formats.

Table 3: Functionality mapping between project functions, Jira user stories, interface storyboards, architecture components, and implementation justification.

Traceability note: Sprint 1 items are H19BBREAD-1 to H19BBREAD-6, while H19BBREAD-7 to H19BBREAD-13 remain in the product backlog and can be prioritised in later sprints based on team velocity and project needs. The Related Storyboard column uses the UI traceability matrix: SCR-01 (FR-1, FR-10, FR-12), SCR-02 (FR-2, FR-3, FR-9), SCR-03 (FR-4, FR-5, FR-6, FR-7), SCR-04 (FR-13), and SCR-05 (FR-8, FR-11).

Bibliography

- 2025 State of the API Report* | Postman. (n.d.). Retrieved June 19, 2026, from <https://www.postman.com/state-of-api/2025/>
- APEC 2025, Interoperability of Electronic Invoicing Systems in the APEC Region, Asia-Pacific Economic Cooperation*. (n.d.). Retrieved June 19, 2026, from https://www.apec.org/docs/default-source/publications/2025/2/225_cti_interoperability-of-electronic-invoicing-systems-in-the-apec-region.pdf
- Australian Peppol Authority* | ATO Software Developers. (n.d.). Retrieved June 19, 2026, from <https://softwaredevelopers.ato.gov.au/australian-peppol-authority>
- Bellon, M., Dabla-Norris, E., Khalid, S., & Lima, F. (2022). Digitalization to improve tax compliance: Evidence from VAT e-Invoicing in Peru. *Journal of Public Economics*, 210, 104661.
- De, B. (2017). API management: an architect's guide to developing and managing APIs for your organization, ed., ser. *For Professionals by Professionals*. Berkeley, CA: Apress.
- Fu, S. D., & Chen, X. (2024). Compound Schema Registry. *Arxiv Preprint Arxiv:2406.11227*.
- Khan, J. Y., Khondaker, M. T. I., Uddin, G., & Iqbal, A. (2021). Automatic detection of five api documentation smells: Practitioners' perspectives. *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 318–329.
- Kotsogiannis, C., Salvadori, L., Karangwa, J., & Murasi, I. (2025). E-invoicing, tax audits and VAT compliance. *Journal of Development Economics*, 172, 103403.
- Rodríguez-Sánchez, M. J., Callejas, Z., Ruiz-Zafra, A., & Benghazi, K. (2025). Spec2chat: A Python library for task-oriented dialogue generation from OpenAPI specifications. *Softwarex*, 32, 102409.
- Supporting business adoption of electronic invoicing - Consultation Paper*. (n.d.).
- Uddin, G., Baysal, O., Guerrouj, L., & Khomh, F. (2019). Understanding how and why developers seek and analyze API-related opinions. *IEEE Transactions on Software Engineering*, 47(4), 694–735.
- VAT Gap - Taxation and Customs Union - European Commission*. (n.d.). Retrieved June 19, 2026, from https://taxation-customs.ec.europa.eu/taxation/vat/fight-against-vat-fraud/vat-gap_en

Appendix A: Scope Approval Evidence

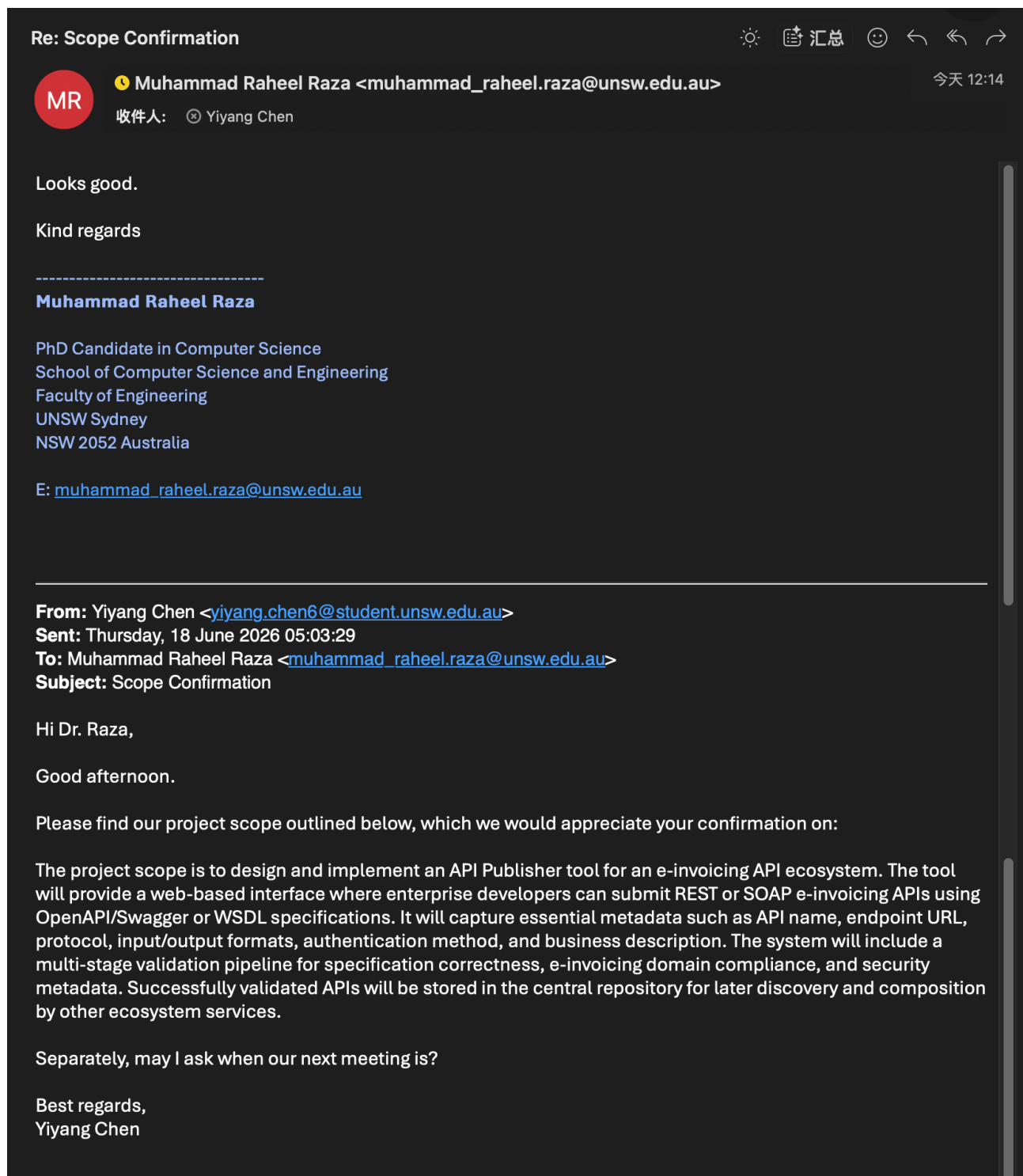


Figure 9: Client's approval for our scope statement